Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г. В. Плеханова» Кафедра информационных технологий и программирования

Пояснительная записка

К пересдаче практической работы по дисциплине: Технология разработки программного обеспечения

Выполнил: Миниханов Вадим Павлович

Группа: ___ИПс-31_____

Проверил: _____

Пермь 2026

Оглавление

1. Введение 2
2. Анализ предметной области 2
2.1. Основные понятия 2
2.2. Сценарии использования 3
2.3. Функциональные и нефункциональные требования 3
3. Архитектура приложения 3
3.1. Паттерн MVVM 3
3.2. Структура решения 4
4. Проектирование модели данных 4
5. Реализация сервисного слоя (Бизнес-логика) 5
5.1. Валидация и целостность данных 5
5.2. Обеспечение транзакционности при проведении накладных 5
5.3. Двухэтапный импорт из CSV 6
6. Реализация пользовательского интерфейса (WPF) 6
6.1. Тема оформления 6
6.2. Изоляция диалоговых окон через IDialogService 6
6.3. Реактивный поиск 6
6.4. Валидация в диалогах 7
7. Тестирование приложения 7
7.1. Настройка тестового проекта 7
7.2. Состав тестов 7
7.3. Прогон тестов 11
8. Ручное тестирование и демонстрация работы 11
9. Заключение 13

1. Введение

Автоматизация складского учета является критически важной задачей для любой организации, занимающейся торговлей или производством. Ручной учет товаров, особенно при наличии нескольких складов и постоянном движении номенклатуры, приводит к ошибкам, потере данных и финансовым убыткам.

Целью данной практической работы является разработка десктопного приложения «Управление складом» с использованием платформы WPF. Система должна обеспечивать ведение справочной информации (организации, склады, товары) и предоставлять инструменты для документирования движения товаров с помощью накладных.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать предметную область и выделить ключевые сущности.
2. Спроектировать архитектуру приложения в соответствии с паттерном MVVM (Model-View-ViewModel).
3. Реализовать модели данных и бизнес-логику (сервисы CRUD) в отдельной, независимой от интерфейса библиотеке классов.
4. Разработать пользовательский интерфейс с единой темной темой оформления.
5. Реализовать логику проведения накладных с автоматическим изменением остатков на складе.
6. Покрыть сервисный слой и ViewModel-и модульными (Unit) тестами.

Технологический стек: язык C#, платформа .NET 8, фреймворк WPF, архитектурный паттерн MVVM, фреймворк тестирования MSTest. В качестве хранилища данных используются коллекции в оперативной памяти (List<T>), что делает систему независимой от СУБД и позволяет легко заменить хранилище в будущем.

2. Анализ предметной области

2.1. Основные понятия

В системе выделены следующие ключевые сущности (иерархия контекста):

- Организация - юридическое лицо. Является корневым элементом. Владеет складами.
- Склад - физическое место хранения товаров. Принадлежит конкретной организации.
- Товар - единица номенклатуры (имеет цену, остаток, скидку, производителя, фото и т.д.). Привязан к конкретному складу.

- Накладная - документ, фиксирующий перемещение товаров. Может быть приходной (поступление) или расходной (списание). Имеет жизненный цикл: Черновик -> Подтверждена.

2.2. Сценарии использования

1. Пользователь запускает приложение и видит список организаций.
2. Выбрав организацию, пользователь переходит в рабочее окно (Модуль складов).
3. В рабочем окне отображаются склады выбранной фирмы. Выбрав склад, пользователь видит таблицу товаров.
4. Пользователь может добавлять/редактировать/удалять склады и товары, а также искать товары по названию.
5. Для изменения остатков пользователь создает накладную, добавляет в нее товары и подтверждает. Остатки на складе мгновенно обновляются.

2.3. Функциональные и нефункциональные требования

Функциональные требования:

- Полноценные CRUD-операции для всех сущностей.
- Каскадное удаление (удаление организации влечет удаление её складов и товаров).
- Защита от некорректного удаления (удаление организации требует ввода её точного названия, а удаление товара блокируется, если он используется в черновике накладной).
- Импорт номенклатуры из CSV-файла.
- Поиск товаров по четырем полям (наименование, категория, производитель, поставщик).

Нефункциональные требования:

- Единая тема оформления, заданная на уровне приложения (Application).
- Валидация ввода на уровне пользовательского интерфейса (запрет пустых полей и отрицательных чисел).
- Вывод понятных сообщений об ошибках бизнес-логики.
- Высокое покрытие кода Unit-тестами.

3. Архитектура приложения

3.1. Паттерн MVVM

Приложение построено на классическом паттерне MVVM. Это обеспечивает чистое разделение ответственности:

- Model (Модели) - описывают структуру данных.
- View (Представления) - XAML-файлы, отвечают только за отрисовку. Не содержат логики.
- ViewModel - связующее звено. Хранит состояние экрана, списки данных и команды (RelayCommand). Не имеет ссылок на визуальные элементы.
- Services (Сервисы) - содержат бизнес-логику (валидация, расчеты).

3.2. Структура решения

Решение WarehouseApp состоит из трех проектов:

1. WarehouseApp.Core - библиотека классов. Содержит Модели, Сервисы, ViewModel-и и Хранилище данных. Не зависит от WPF.
2. WarehouseApp.Wpf - графический интерфейс (Окна, Диалоги, Стили, Тема).
3. WarehouseApp.Tests - проект с модульными тестами. Ссылается только на Core, что позволяет тестировать логику без запуска графической оболочки.

4. Проектирование модели данных

Все модели используют целочисленные идентификаторы (int Id) для простоты генерации и отладки.

```
namespace WarehouseApp.Core.Models;
public class Product
{
   public int Id { get; set; }
   public int OrganizationId { get; set; }
   public int WarehouseId { get; set; }
   public string? PhotoPath { get; set; }
   public string Name { get; set; } = string.Empty;
   public string Category { get; set; } = string.Empty;
   public string Manufacturer { get; set; } = string.Empty;
   public string Supplier { get; set; } = string.Empty;
   public decimal Price { get; set; }
```

```
    public int Stock { get; set; }
    public decimal DiscountPercent { get; set; }
    public decimal PriceWithDiscount => Math.Round(Price * (1 - DiscountPercent / 100m), 2);
}
```

Объяснение к коду:Модель Product содержит все 8 обязательных колонок, требуемых по заданию (Наименование, Категория, Производитель, Поставщик, Цена, Остаток, Скидка % и Путь к фото). Использование типа decimal для цены предотвращает ошибки округления, свойственные типу float/double при работе с деньгами. Также реализовано вычисляемое свойство PriceWithDiscount, которое автоматически рассчитывает цену со скидкой, избавляя от необходимости делать это вручную в интерфейсе.

Хранилище данных реализовано в классе InMemoryDataStore.

```
using WarehouseApp.Core.Models;
namespace WarehouseApp.Core.Data;
public sealed class InMemoryDataStore
{
    private static readonly Lazy<InMemoryDataStore> _instance = new(() => new InMemoryDataStore());
    public static InMemoryDataStore Instance => _instance.Value;
    public List<Organization> Organizations { get; } = new();
    public List<Warehouse> Warehouses { get; } = new();
    public List<Product> Products { get; } = new();
    public List<Invoice> Invoices { get; } = new();
    public int NextOrganizationId { get; set; } = 1;
    public int NextWarehouseId { get; set; } = 1;
    public int NextProductId { get; set; } = 1;
    public int NextInvoiceId { get; set; } = 1;
    public InMemoryDataStore()
    {
        SeedTestData();
    }
    private static string Photo(string fileName) => Path.Combine(AppContext.BaseDirectory, "Images", fileName);
    public void SeedTestData()
    {
```

```
        Organizations.Clear();
        Warehouses.Clear();
        Products.Clear();
        Invoices.Clear();
      var org1 = new Organization { Id = NextOrganizationId++, Name = "Рога и копыта, ООО" };
        var org2 = new Organization { Id = NextOrganizationId++, Name = "Пупкин и сыновья, ООО" };
        Organizations.Add(org1);
        Organizations.Add(org2);
        var wh1 = new Warehouse { Id = NextWarehouseId++, OrganizationId = org1.Id, Name = "Склад Рогов и Копыта №1", Address = "г. Москва, ул. Складская, д. 1" };
        var wh2 = new Warehouse { Id = NextWarehouseId++, OrganizationId = org1.Id, Name = "Склад Рогов и Копыта №2", Address = "г. Москва, ул. Складская, д. 2" };
        var wh3 = new Warehouse { Id = NextWarehouseId++, OrganizationId = org2.Id, Name = "Склад Пупкина №1", Address = "г. Санкт-Петербург, пр. Невский, д. 10" };
        Warehouses.Add(wh1);
        Warehouses.Add(wh2);
        Warehouses.Add(wh3);
        Products.Add(new Product
        {
            Id = NextProductId++,
            OrganizationId = org1.Id,
            WarehouseId = wh1.Id,
            Name = "Молоток слесарный",
            PhotoPath = Photo("hammer.png"),
            Category = "Инструменты",
            Manufacturer = "ЗИЛ",
            Supplier = "ООО ИнструментТорг",
            Price = 450m,
            Stock = 25,
            DiscountPercent = 0
        });
        Products.Add(new Product
        {
```

```
    Id = NextProductId++,
    OrganizationId = org1.Id,
    WarehouseId = wh1.Id,
    Name = "Отвёртка крестовая",
    PhotoPath = Photo("screwdriver.png"),
    Category = "Инструменты",
    Manufacturer = "Stanley",
    Supplier = "ООО ИнструментТорг",
    Price = 180m,
    Stock = 60,
    DiscountPercent = 10
});
Products.Add(new Product
{
    Id = NextProductId++,
    OrganizationId = org1.Id,
    WarehouseId = wh2.Id,
    Name = "Кабель ВВГ 3х2.5",
    PhotoPath = Photo("cable.png"),
    Category = "Электрика",
    Manufacturer = "Камкабель",
    Supplier = "ООО ЭлектроСнаб",
    Price = 75.5m,
    Stock = 500,
    DiscountPercent = 5
});
Products.Add(new Product
{
    Id = NextProductId++,
    OrganizationId = org2.Id,
    WarehouseId = wh3.Id,
    Name = "Бумага офисная А4",
    PhotoPath = Photo("paper.png"),
    Category = "Канцелярия",
    Manufacturer = "SvetoCopy",
```

```
            Supplier = "ООО КанцОпт",
            Price = 320m,
            Stock = 150,
            DiscountPercent = 0
        });
    }
}
```

Объяснение к коду:Класс реализован по паттерну «Одиночка» (Singleton) через Lazy<T> — это гарантирует, что в приложении существует только один экземпляр базы данных, и все окна работают с одними и теми же данными. Данные хранятся в типизированных коллекциях List<T>. Важным архитектурным решением является наличие публичного конструктора: это позволяет в тестах создавать изолированные экземпляры хранилища (new InMemoryDataStore()), чтобы тесты не влияли друг на друга.

5. Реализация сервисного слоя (Бизнес-логика)

Все операции с данными инкапсулированы в сервисах (OrganizationService, WarehouseService, ProductService, InvoiceService). При нарушении бизнес-правил сервисы выбрасывают кастомное исключение ValidationException.

5.1. Валидация и целостность данных

В OrganizationService реализовано удаление с подтверждением и очисткой связанных данных.

```
public void Delete(int id, string confirmationName)
{
    var org = GetById(id);
    if (!string.Equals((confirmationName ?? string.Empty).Trim(), org.Name, StringComparison.Ordinal))
        throw new ValidationException("Введённое название не совпадает с названием организации. Удаление отменено.");
    var warehouseIds = _store.Warehouses.Where(w => w.OrganizationId == id).Select(w => w.Id).ToList();
    foreach (var whId in warehouseIds)
    {
        _store.Products.RemoveAll(p => p.WarehouseId == whId);
        _store.Invoices.RemoveAll(inv => inv.WarehouseId == whId);
```

```
        }
        _store.Warehouses.RemoveAll(w => w.OrganizationId == id);
        _store.Organizations.Remove(org);
    }
}
```

Объяснение к коду:Метод Delete требует передачи подтверждающей строки. Если пользователь ввел название с ошибкой (даже в регистре), выбрасывается исключение, и удаление отменяется. Если текст совпадает, система выполняет каскадное удаление: сначала собираются ID всех складов организации, затем удаляются все товары и накладные, привязанные к этим складам, и только потом удаляются сами склады и организация. Это предотвращает появление «висячих» данных (Data Integrity).

5.2. Обеспечение транзакционности при проведении накладных

Метод подтверждения накладной в InvoiceService реализует принцип атомарности («всё или ничего»).

```
public void Confirm(int invoiceId)
{
    var invoice = GetById(invoiceId);
    EnsureDraft(invoice);

    if (invoice.Items.Count == 0)
        throw new ValidationException("Нельзя подтвердить пустую накладную без товарных позиций.");

    if (invoice.Type == InvoiceType.Outgoing)
    {
        foreach (var item in invoice.Items)
        {
            var product = _productService.GetById(item.ProductId);
            if (product.Stock < item.Quantity)
                throw new ValidationException(
                    $"Недостаточно товара \"{product.Name}\" на складе: в наличии {product.Stock}, требуется {item.Quantity}.");
        }
    }
```

```
            foreach (var item in invoice.Items)
            {
                var product = _productService.GetById(item.ProductId);
                product.Stock += invoice.Type == InvoiceType.Incoming ? item.Quantity : -
item.Quantity;
            }

            invoice.Status = InvoiceStatus.Confirmed;
        }
```

Объяснение к коду:Логика разделена на два этапа. Сначала система проверяет, не пуста ли накладная. Затем, если накладная расходная, запускается цикл предварительной проверки остатков. Если хотя бы одного товара на складе не хватает (product.Stock < item.Quantity), выбрасывается исключение, и выполнение метода прерывается. Изменение остатков (цикл с прибавлением или вычитанием) происходит только после успешной проверки всех позиций. Это гарантирует, что при нехватке хотя бы одного товара, остатки остальных товаров не будут испорчены.

5.3. Двухэтапный импорт из CSV

Метод ImportFromCsv в ProductService реализован в два этапа. Сначала файл читается построчно и парсится. Каждая строка валидируется (проверяется количество колонок, формат чисел). Все успешно распарсенные строки временно сохраняются в список.
И только если ошибок нет - происходит фактическое добавление товаров в базу (второй цикл). Это защищает систему от частичного импорта «битого» файла. Для парсинга чисел используется CultureInfo.InvariantCulture, что позволяет корректно читать десятичные дроби как с точкой, так и с запятой.

6. Реализация пользовательского интерфейса (WPF)

6.1. Тема оформления

Дизайн приложения выдержан в темных тонах. Цвета и стили вынесены в отдельный файл DarkTheme.xaml и подключены глобально в App.xaml. Переопределены шаблоны для кнопок, текстовых полей, таблиц (DataGrid) и контекстных меню. Это гарантирует, что текст не сольется с фоном (например, при выпадающих списках).

6.2. Изоляция диалоговых окон через IDialogService

В WPF вызов MessageBox.Show() или открытие нового окна напрямую из ViewModel нарушает принцип тестируемости. Для решения этой проблемы используется интерфейс IDialogService. ViewModel-и не знают, как именно отображать диалог; они просто вызывают метод (например, ShowProductDialog).

В реальном приложении внедряется класс DialogService, открывающий окна WPF. В тестах же внедряется MockDialogService, который просто возвращает заранее заданные значения («Да» для подтверждения, готовый объект товара и т.д.). Это позволяет тестировать логику работы кнопок без запуска интерфейса.

6.3. Реактивный поиск

Свойство SearchText в WorkspaceViewModel реализовано с паттерном реактивности. В сеттере свойства при каждом изменении текста вызывается метод ReloadProducts(). Так как в XAML для TextBox установлено UpdateSourceTrigger=PropertyChanged, база данных и таблица DataGrid перерисовываются мгновенно при каждом нажатии клавиши пользователем. Это обеспечивает быстрый UX без необходимости нажимать кнопку «Найти».

6.4. Валидация в диалогах

При редактировании товара (ProductEditDialog) реализована клиентская валидация. При вводе букв в поле цены или пустом наименовании, кнопка «ОК» не закрывает окно, поле подсвечивается красной рамкой, а под формой выводится текст ошибки. Ошибки бизнес-логики (исключения) перехватываются во ViewModel и выводятся пользователю в виде MessageBox.

7. Тестирование приложения

7.1. Настройка тестового проекта

Тесты написаны с использованием фреймворка MSTest. Важное архитектурное решение: класс InMemoryDataStore имеет публичный конструктор. Это позволяет в методе TestInitialize создавать изолированный

экземпляр хранилища (new InMemoryDataStore()) для каждого теста. Каждый тест работает с чистой базой данных, заполненной тестовыми данными через SeedTestData(). Результаты одного теста не влияют на другой.

7.2. Состав тестов

В проекте WarehouseApp.Tests реализовано 89 модульных тестов. Тестирование разбито на три уровня: проверка инфраструктуры, проверка бизнес-логики (Сервисы) и проверка поведения интерфейса (ViewModel). Особое внимание уделено негативным сценариям - проверке того, как система защищается от неверных действий пользователя.

Группа тестов (Файл)	Проверяемые сценарии	Что именно проверяется	Зачем это нужно (Бизнес-ценность)
Инфраструктура (RelayCommandTests)	Execute_InvokesAction; CanExecute_UsesPredicate	Вызов методов при нажатии кнопок. Проверка того, что кнопка блокируется (становится серой), если условие CanExecute возвращает false.	Базовая надежность интерфейса. Если команды работают неправильно, кнопки в приложении будут «мертвыми» или будут вызывать ошибки.
Сервисы: Организации (OrganizationServiceTests)	Add_DuplicateName_Throws; Delete_WrongConfirmation_Throws; Delete_CascadesToWarehousesAndProducts	Запрет на добавление двух фирм с одинаковым названием. Проверка ввода точного названия при удалении. Проверка того, что при удалении фирмы стираются	Целостность данных. Предотвращение появления «мусорных» данных и «висячих» складов, которые остались без владельца.

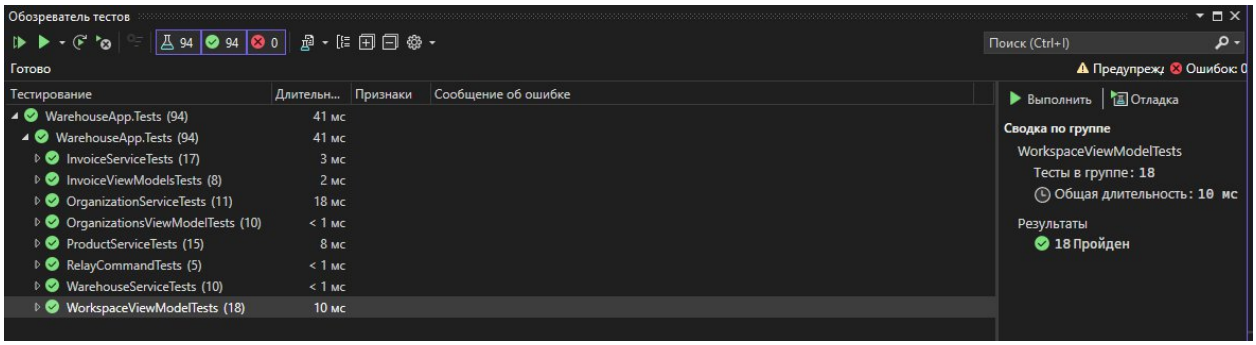
Группа тестов (Файл)	Проверяемые сценарии	Что именно проверяется	Зачем это нужно (Бизнес-ценность)
		все её склады и товары.	
Сервисы: Склады (WarehouseServiceTests)	Add_EmptyName_Throws; Add_DuplicateNameWithinOrganization_Throws; Delete_WarehouseWithProducts_CascadesProductsAndInvoices	Запрет пустых имен. Запрет дублей в рамках одной фирмы. Проверка каскадного удаления товаров и накладных при удалении склада.	Защита от потери номенклатуры. Логика: нельзя удалить склад, если на нем лежат товары (или удалить их вместе с ним безопасно для системы).
Сервисы: Товары (ProductServiceTests)	Add_NegativePrice_Throws; Add_DiscountOutOfRange_Throws; ImportFromCsv_ValidFile; ImportFromCsv_InvalidRow	Проверка цен (нельзя отрицательные). Проверка скидки (строго 0-100). Тест импорта CSV: успешный парсинг и откат всей операции, если в одной из строк ошибка.	Финансовая безопасность и атомарность. Если пользователь загружает файл из 100 строк, и в 50-й опечатка - система не должна загрузить первые 49 строк, а должна отменить всё.
Сервисы: Накладные (InvoiceServiceTests)	Confirm_IncomingInvoice_IncreasesStock; Confirm_OutgoingInvoice_InsufficientStock_DoesNotChangeAnyStock	Проверка изменения остатков. Главный тест - проверка транзакционности: если в расходной накладной 3	Главная бизнес-логика. Защита от ухода остатков в минус. Гарантия того, что документ либо проводится целиком, либо не проводится вообще (принцип «всё или

Группа тестов (Файл)	Проверяемые сценарии	Что именно проверяется	Зачем это нужно (Бизнес-ценность)
		товара, и одного не хватает, остатки остальных двух тоже НЕ должны измениться.	ничего»).
ViewModel: Организации (OrganizationsViewModelTests)	Commands_RequireSelection; Add_EmptyName_ShowsError; Open_PassesSelectedOrganizationAsContext	Проверка блокировки кнопок (нельзя нажать «Удалить», если ничего не выделено). Проверка вывода ошибок через моки. Проверка передачи контекста (ID фирмы) в рабочее окно.	UX/UI логика. Пользователь не должен иметь возможности нажать кнопку, которая приведет к ошибке. Корректная навигация между окнами.
ViewModel: Рабочее окно (WorkspaceViewModelTests)	ContextStatus_ContainsOrganizationAndWarehouse; SearchText_FiltersProducts; DeleteWarehouse_Cancelled_KeepsWarehouse	Проверка строки состояния (видит ли пользователь, где он). Проверка фильтрации списка при вводе текста. Проверка отмены действия (если нажали «Нет» в диалоге).	Навигация и удобство. Пользователь всегда видит, в каком складе он находится. Поиск работает мгновенно при вводе символов.
ViewModel:	Editor_ConfirmOu	Связка	Связность слоев.

Группа тестов (Файл)	Проверяемые сценарии	Что именно проверяется	Зачем это нужно (Бизнес-ценность)
Накладные (InvoiceViewModels Tests)	tgoing_Insufficient Stock_ShowsError ; List_Confirm_App liesStockChange	интерфейса и логики: когда сервис кидает ошибку (нехватка товара), ViewModel перехватывает её и показывает пользователю. Проверка того, что после подтверждения цифры в таблице обновились.	Проверка того, что нажатие кнопок в UI реально приводит к изменениям в бизнес-логике и обновлению экрана.

7.3. Прогон тестов

Запуск осуществляется через обозреватель тестов Visual Studio. Все 94 теста проходят успешно



8. Ручное тестирование и демонстрация работы

В ходе ручного тестирования были проверены основные сценарии:

1. Создание новой организации и её переименование.
2. Удаление организации с вводом подтверждающего текста.
3. Создание складов внутри организации.
4. Добавление товаров, загрузка фотографий, проверка валидации (попытка ввести буквы в поле цены).

5. Импорт товаров из CSV-файла.

6. Создание накладной, добавление товаров и подтверждение. Проверено, что остатки в таблице обновились.

7. Попытка списать товара больше, чем есть на складе (система выдала ошибку, остатки не изменились).

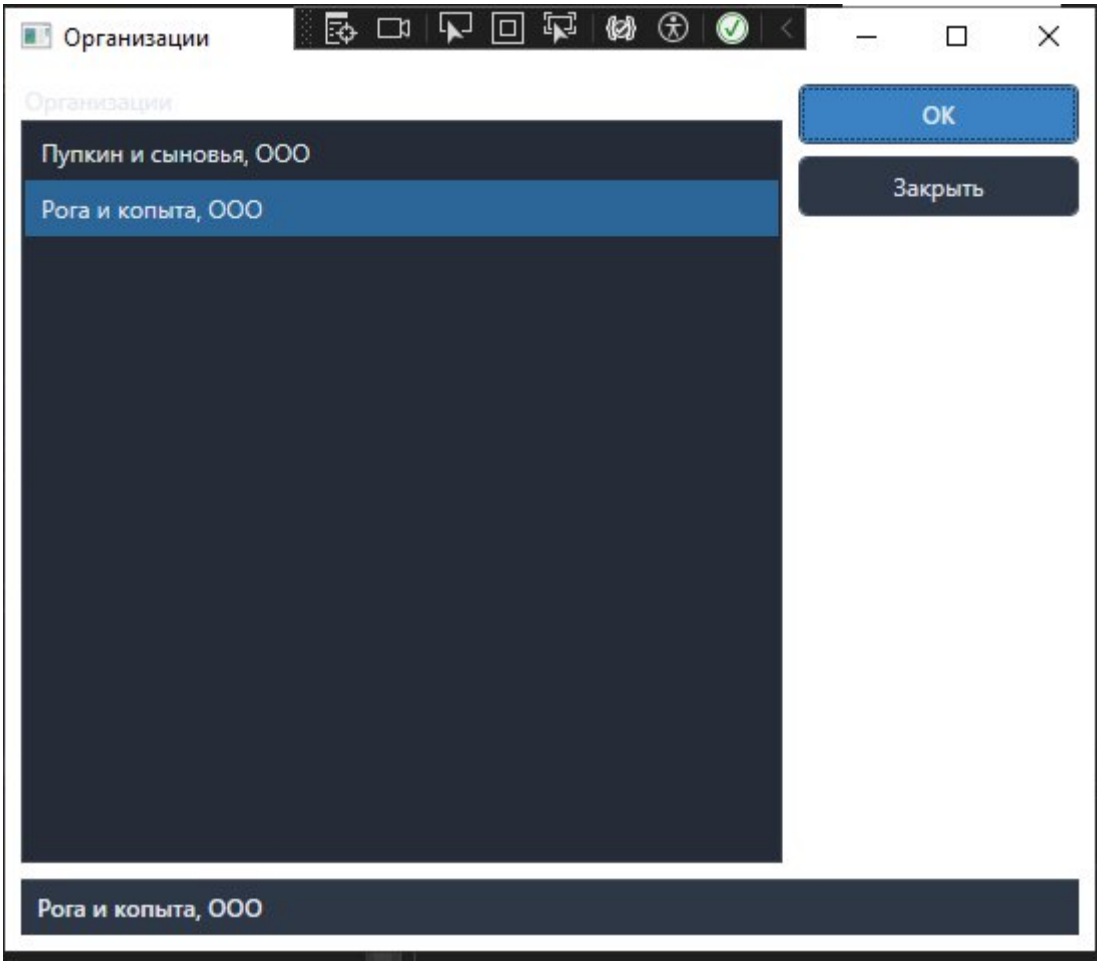


Рис.1 Список Организаций

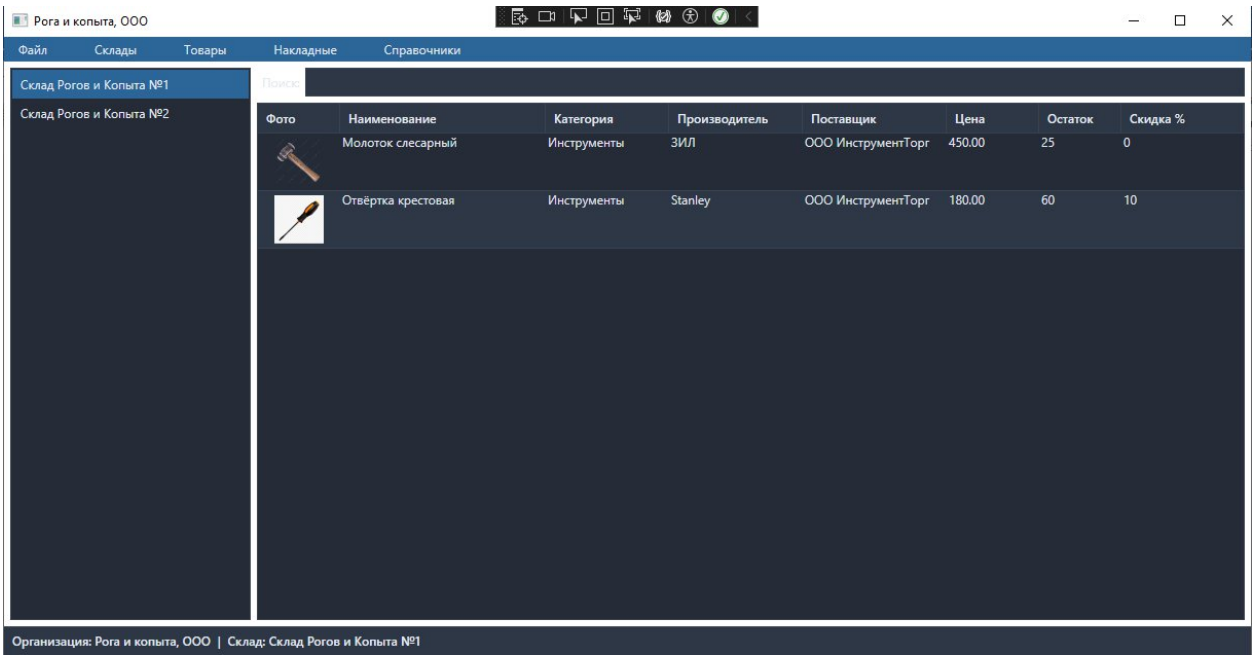


Рис.2 Список товаров

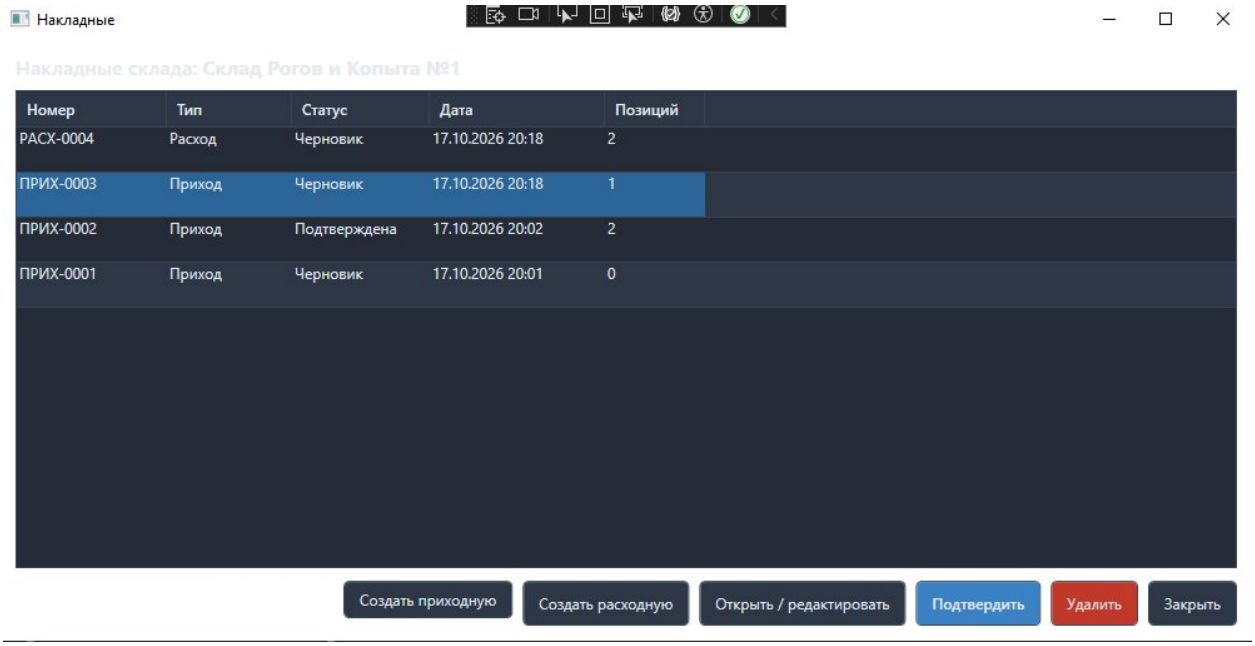


Рис.3 Список накладных

9. Заключение

В ходе работы было разработано WPF-приложение, полностью соответствующее техническому заданию. Были реализованы:

- Три справочных модуля (Организации, Склады, Товары).
- Полноценный модуль накладных с логикой прихода/расхода и защитой от отрицательных остатков.

- Импорт данных из CSV.
- Темная тема оформления.
- 94 модульных тестов, покрывающих как сервисы, так и ViewModel-и.